

목차

1장 조회가 느리다는 말	2
1.1 느림을 무엇으로 재는가	2
1.2 평균과 꼬리	3
1.3 어디서 시간이 드는가	4
1.4 조회 한 건과 화면 한 장	5
1.5 무엇을 먼저 재는가	6
1.6 시간이 흠어지는 자리	7
1.7 빠른 조회의 기준	8
2장 저장된 자료를 찾는 방법	9
2.1 전체를 훑는 조회	9
2.2 색인이 하는 일	10
2.3 색인의 구조와 정렬	11
2.4 색인이 쓰이지 않는 조건	12
2.5 여러 항목을 묶은 색인	13
2.6 훑기와 색인 찾기	14
2.7 색인이 늘리는 비용	15
3장 실행 계획 읽기	16
3.1 실행 계획이란	16
3.2 계획에서 볼 세 가지	17
3.3 예상과 실제의 차이	18
3.4 계획이 바뀌는 이유	19
3.5 같은 조회의 두 계획	20
3.6 계획을 바꾸는 방법	21
3.7 계획만 보고 판단할 때	22
4장 반복 조회 문제	23
4.1 한 번이 여러 번이 되는 자리	23
4.2 목록과 상세의 관계	24
4.3 한꺼번에 가져오기	25
4.4 반복 조회가 늘어나는 모양	26
4.5 필요한 만큼만 가져오기	27
4.6 캐시를 두기 전에	28
4.7 반복을 세는 방법	29
5장 접근 패턴 관찰	30
5.1 무엇을 관찰할 것인가	30
5.2 느린 조회 모으기	31
5.3 자주 오는 조회	32
5.4 조회의 두 갈래 분포	33
5.5 자료가 늘면 달라지는 것	34
5.6 실제와 시험 환경의 차이	35
5.7 관찰이 부담이 될 때	36
6장 느린 화면 하나를 고친 기록	37
6.1 느린 화면의 증상	37
6.2 조회 횟수를 세어 본 결과	38
6.3 반복을 줄인 과정	39
6.4 남은 조회의 실행 계획	40
6.5 색인을 정한 과정	41
6.6 고친 뒤 다시 켜 본 결과	42
6.7 남긴 장치와 다음 후보	43
7장 성능 판단 기준	44
7.1 어디부터 손댈 것인가	44
7.2 고치기 전에 재기	45
7.3 색인을 더할 때와 지울 때	46
7.4 성능 점검 항목	47
7.5 구조를 바꾸는 선택	48
7.6 성능과 다른 목표의 균형	49
7.7 핵심 원칙 정리	50

1.1 느림을 무엇으로 재는가

느리다는 말은 그대로 다룰 수 없다. 다루려면 수로 바꿔야 하고, 가장 기본이 되는 수는 조회 하나가 시작해서 끝날 때까지 걸린 시간이다. 이 수가 없으면 고친 뒤에 나아졌는지도 말할 수 없다.

재는 자리도 함께 정해야 한다. 저장소가 답을 만드는 데 걸린 시간과, 부르는 쪽이 답을 받기까지 걸린 시간은 다르다. 뒤쪽에는 연결을 얻는 시간과 자료를 옮기는 시간이 들어 있다. 어느 쪽을 말하는지 정하지 않으면 같은 조회를 두고 서로 다른 수를 놓고 이야기하게 된다.

이 책은 조회 한 건이 저장소에서 처리되는 시간과, 그 조회가 화면 하나를 만드는 데 몇 번 일어나는지를 함께 본다. 한 건이 빨라도 여러 번 일어나면 느리고, 한 번만 일어나도 그 한 건이 오래 걸리면 느리다. 두 수를 나눠 보는 것이 진단의 출발이다.

1.2 평균과 꼬리

응답 시간을 하나의 수로 줄이면 평균을 쓰기 쉽지만, 평균은 느린 쪽을 감춘다. 백 번 가운데 아흔아홉 번이 빠르고 한 번이 아주 느리면 평균은 여전히 빠르게 보인다. 그러나 그 한 번을 겪은 사람에게는 이 조회가 느린 조회다.

그래서 분포를 본다. 가운뎃값과 함께 상위 몇 퍼센트의 값을 보면 꼬리가 얼마나 긴지 드러난다. 꼬리가 길다는 것은 같은 조회가 조건에 따라 크게 다르게 처리된다는 뜻이고, 대개 여기에 원인이 숨어 있다.

꼬리를 볼 때는 그 조회들이 무엇이 다른지 함께 본다. 특정 사용자, 특정 기간, 특정 값에서만 느리다면 그 조건이 곧 단서다. 평균은 상태를 알려 주고 꼬리는 원인을 가리킨다. 진단에 쓰는 것은 대개 뒤쪽이다.

1.3 어디서 시간이 드는가

조회 하나가 걸리는 시간은 여러 자리에 나뉘어 있다. 저장소가 자료를 찾는 시간, 찾은 자료를 정렬하거나 묶는 시간, 결과를 옮기는 시간, 그리고 부르는 쪽이 그 결과를 다루는 시간이다.

어느 자리에 시간이 몰려 있는지에 따라 고칠 방법이 완전히 다르다. 찾는 데 오래 걸리면 색인을 보고, 옮기는 데 오래 걸리면 가져오는 양을 보고, 부르는 쪽에서 오래 걸리면 조회 횟수를 본다. 자리를 정하지 않고 색인부터 손대는 것이 가장 흔한 헛수고다.

자리를 나누려면 최소한 두 수가 필요하다. 저장소에서 걸린 시간과 전체 걸린 시간이다. 둘의 차이가 크면 저장소 밖에 원인이 있다. 이 두 수만 있어도 볼 자리가 절반으로 준다.

1.4 조회 한 건과 화면 한 장

조회 한 건이 빠르다는 사실은 화면 한 장이 빠르다는 뜻이 아니다. 화면 하나를 그리는 데 조회가 쉰 번 일어나면 각각이 아무리 빨라도 합은 크다. 반대로 조회 한 건이 느려도 화면에서 한 번만 일어나면 전체에서 차지하는 몫이 작을 수 있다.

그래서 성능을 볼 때는 늘 단위를 밝힌다. 이 수가 조회 한 건의 수인지, 요청 한 번의 수인지, 화면 한 장의 수인지에 따라 판단이 달라진다. 단위를 밝히지 않은 수는 견줄 수 없다.

실무에서 크게 개선되는 경우는 대개 한 건을 빠르게 만든 것이 아니라 횟수를 줄인 것이다. 쉰 번을 한 번으로 줄이면 한 건의 시간을 절반으로 줄이는 것보다 훨씬 크다. 4장은 이 횟수 문제를 따로 다룬다.

1.5 무엇을 먼저 재는가

성능 작업은 재는 것에서 시작한다. 재지 않고 고치면 좋아졌는지 알 수 없고, 나빠져도 모른다. 고친 뒤에 재는 것으로는 부족하다. 견줄 기준이 없기 때문이다.

재는 순서는 넓은 데서 좁은 데로 간다. 먼저 어떤 요청이 느린지 보고, 그 요청 안에서 어떤 조회가 시간을 많이 쓰는지 보고, 그 조회가 몇 번 일어나는지 본다. 이 순서를 밟으면 대개 세 번째 단계에서 원인이 드러난다.

재는 것 자체에도 비용이 든다. 모든 조회의 시간을 남기면 그 기록이 부담이 되므로, 느린 것만 남기거나 표본을 남긴다. 다만 진단하는 동안에는 넉넉히 남기고 끝난 뒤에 줄이는 편이 낫다. 없는 기록은 나중에 만들 수 없다. 재는 장치를 갖추는 데 든 시간은 다음 진단에서 그대로 돌아온다.

시간이 흠어지는 자리

어느 칸이 넓은지에 따라 볼 자리가 다르다

요청 한 번

찾기	목록 정렬하기	옮기기	부르는 쪽 처리
색인	가져오는 양	결과 크기	조회 횟수

조회 한 건 × 여러 번

한 건은 빠르는데 합이 크다

같은 시간을 두 가지로 나눠 보아야 한다

1.7 빠른 조회의 기준

성능 작업에는 끝이 없다. 어디까지 할지 정하지 않으면 계속 손대게 되고, 그 사이에 다른 일이 밀린다. 그래서 시작할 때 목표를 수로 정한다. 이 화면이 얼마 안에 뜨면 되는지, 상위 몇 퍼센트를 기준으로 삼을지까지 적는다.

목표는 사용자의 경험에서 온다. 사람이 기다린다고 느끼는 시점, 다른 작업으로 옮겨 가는 시점이 기준이 된다. 저장소가 낼 수 있는 최선에서 목표를 정하면 필요 없는 곳까지 갈고닥게 된다.

목표를 넘긴 뒤에는 멈춘다. 더 빠르게 만드는 일은 대개 구조를 복잡하게 만들고, 복잡해진 구조는 다음 사람의 시간을 쓴다. 성능은 다다익선이 아니라 목표를 넘겼는지로 판단하는 값이다. 목표를 적어 두면 언제 멈출지도 함께 정해진다.

2.1 전체를 훑는 조회

저장소가 조건에 맞는 자료를 찾는 가장 단순한 방법은 처음부터 끝까지 읽으며 맞는지 확인하는 것이다. 이 방식은 자료가 적을 때는 충분히 빠르고, 조건이 까다로워 어차피 대부분을 읽어야 할 때도 나쁘지 않다.

문제는 자료가 늘어날 때 시간도 함께 늘어난다는 점이다. 열 배가 되면 시간도 대략 열 배가 된다. 처음에는 빠르던 화면이 몇 달 뒤에 느려지는 대부분의 경우가 이 모양이다. 코드는 그대로인데 자료만 늘었다.

그래서 전체 훑기가 나쁜 것이 아니라, 자료가 계속 늘어나는 자리에서 전체 훑기가 남아 있는 것이 문제다. 지금 빠른지가 아니라 자료가 열 배가 되어도 견디는지로 판단해야 한다. 다음 절의 색인은 이 늘어남을 끊기 위한 장치다.

2.2 색인이 하는 일

색인은 특정 항목을 기준으로 자료의 위치를 미리 정리해 둔 별도의 구조다. 책 뒤의 찾아보기와 같은 역할을 한다. 조건에 맞는 자료를 찾을 때 전체를 읽는 대신 정리된 순서를 따라 바로 그 자리로 간다.

색인이 있으면 자료가 늘어도 찾는 시간이 크게 늘지 않는다. 정리된 구조에서는 몇 번의 비교만으로 범위가 좁혀지기 때문이다. 자료가 열 배가 되어도 비교 횟수는 조금만 늘다. 이 성질이 색인을 쓰는 이유의 거의 전부다.

다만 색인은 공짜가 아니다. 저장할 자리를 더 쓰고, 자료를 넣거나 고칠 때마다 색인도 함께 고쳐야 한다. 그래서 색인은 많을수록 좋은 것이 아니라 조회 방식에 맞는 것만 두는 것이 맞다. 이 대가는 2.7에서 따로 다룬다.

2.3 색인의 구조와 정렬

색인의 힘은 정렬에서 온다. 값이 순서대로 놓여 있으면 특정 값을 찾는 일뿐 아니라 어떤 값보다 큰 것들을 모으는 일, 정렬된 결과를 그대로 내보내는 일까지 빨라진다.

그래서 색인은 세 가지 일에 함께 쓰인다. 값이 같은 자료 찾기, 값의 범위에 드는 자료 찾기, 그리고 그 순서로 결과를 내놓기다. 조회가 정렬을 요구할 때 색인의 순서와 맞으면 정렬하는 시간이 아예 사라진다.

반대로 색인의 순서와 조회가 원하는 순서가 다르면 색인으로 찾은 뒤 다시 정렬해야 한다. 이때는 찾기는 빨라도 정렬이 남는다. 색인을 만들 때 어떤 순서로 만들지 정하는 일이 어떤 항목에 만들지 정하는 일만큼 중요한 이유가 여기 있다.

2.4 색인이 쓰이지 않는 조건

색인을 만들어 두었는데 조회가 여전히 느리다면 그 색인이 쓰이지 않았을 가능성이 크다. 쓰이지 않는 전형은 몇 가지로 정해져 있다.

첫째는 조건에 항목을 그대로 쓰지 않고 계산해서 쓰는 경우다. 값을 가공한 결과로 조건을 걸면 미리 정리해 둔 순서가 무의미해진다. 둘째는 조건에 걸리는 자료가 전체의 대부분인 경우다. 이때는 색인을 거쳐 가는 것이 오히려 느려서 저장소가 스스로 전체 훑기를 고른다.

셋째는 색인의 앞부분을 쓰지 않고 뒷부분만 조건에 거는 경우다. 정렬은 앞에서부터 이루어지므로 앞이 정해지지 않으면 순서를 따라갈 수 없다. 세 가지 모두 색인을 더 만들어서 풀 문제가 아니라 조회나 색인의 모양을 맞춰야 풀리는 문제다.

2.5 여러 항목을 묶은 색인

조건이 여러 항목에 걸리면 항목을 묶어 하나의 색인으로 만들 수 있다. 이때 항목의 순서가 성질을 정한다. 앞에 놓인 항목으로 먼저 정렬되고 그 안에서 다음 항목으로 정렬되기 때문이다.

그래서 앞 항목을 조건에 쓰지 않으면 뒤 항목만으로는 순서를 따라갈 수 없다. 묶은 색인을 만들 때는 조회가 늘 조건에 거는 항목을 앞에 두고, 범위로 거는 항목을 뒤에 두는 것이 기본이다.

묶은 색인 하나가 여러 개의 낱개 색인보다 나은 경우가 많다. 낱개 색인은 각각으로 찾은 결과를 다시 맞춰야 하지만, 묶은 색인은 한 번에 좁힌다. 대신 조합마다 색인을 만들면 개수가 빠르게 늘어 갱신 비용이 커진다. 그래서 조합을 늘리기 전에 기존 색인의 앞 항목으로 덮을 수 있는지 먼저 본다.

훅기와 색인 찾기

자료가 늘 때 두 방식이 갈린다

전체 훅기

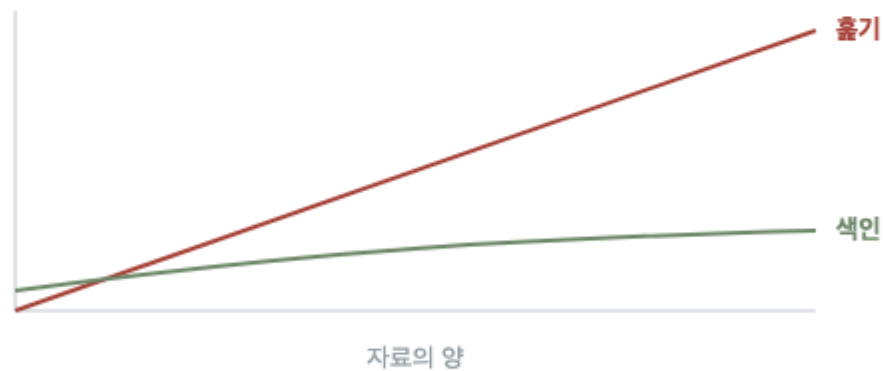


모든 칸을 확인한다

색인 찾기



한 칸만 확인한다



지금 빠르다가 아니라 늘어도 견디는가를 본다

2.7 색인이 늘리는 비용

색인은 조회를 빠르게 하지만 쓰기를 느리게 한다. 자료를 하나 넣을 때마다 그 자료에 걸린 모든 색인을 함께 고쳐야 하기 때문이다. 색인이 다섯 개면 쓰기 한 번에 다섯 번의 정리가 따라온다.

저장 공간도 는다. 색인은 원래 자료와 별도로 저장되므로 항목이 크거나 자료가 많으면 색인이 본체만큼 커지기도 한다. 여기에 더해 색인이 많으면 저장소가 어느 색인을 쓸지 고르는 판단도 복잡해진다.

그래서 색인을 더할 때는 어떤 조회를 위한 것인지 적어 둔다. 적어 두지 않으면 몇 달 뒤에 지워도 되는지 아무도 모른다. 쓰이지 않는 색인은 이득 없이 비용만 남기지만, 근거가 없으면 지우는 판단을 내릴 수 없다. 근거가 적힌 색인만이 나중에 지울 수 있는 색인이다.

3.1 실행 계획이란

같은 조회라도 처리하는 방법은 여럿이다. 어떤 색인을 쓸지, 어느 자료를 먼저 읽을지, 어떻게 묶을지에 따라 걸리는 시간이 크게 달라진다. 저장소는 이 가운데 하나를 골라 실행하며, 고른 절차를 실행 계획이라 한다.

계획은 자료의 분포와 크기에 대한 짐작을 바탕으로 세워진다. 어떤 조건에 몇 건이 걸릴지 짐작해 가장 싸 보이는 절차를 고른다. 그래서 계획은 조회문만 보고 정해지는 것이 아니라 그때의 자료 상태에 따라 달라진다.

성능 진단에서 계획을 보는 이유는 결과가 아니라 과정을 알려 주기 때문이다. 느리다는 사실은 시간이 알려 주지만 왜 느린지는 계획이 알려 준다. 색인을 썼는지, 전체를 훑었는지, 어디서 정렬이 일어났는지가 여기 적힌다.

3.2 계획에서 볼 세 가지

계획은 자세히 읽으면 복잡하지만 진단에서 먼저 볼 것은 셋이다. 첫째, 어떤 방식으로 자료를 찾았는가. 전체를 훑었는지 색인을 썼는지가 여기서 드러난다.

둘째, 각 단계에서 다룬 자료가 몇 건인가. 최종 결과는 열 건인데 중간에서 십만 건을 다뤘다면 그 사이에서 대부분이 걸러진 것이고, 그 거르기를 앞으로 당길 수 있는지 보아야 한다.

셋째, 정렬이나 묶기가 어디서 일어났는가. 이 작업들은 자료를 모두 모은 뒤에 하므로 다루는 건수가 크면 그 자체로 오래 걸린다. 세 가지만 봐도 대부분의 느린 조회는 어느 갈래인지 정해진다. 이 셋은 어느 저장소의 계획에서나 이름만 다를 뿐 같은 자리에 적혀 있다. 넷째 이후의 항목은 이 셋으로 갈래를 정한 뒤에 읽어도 늦지 않다.

3.3 예상과 실제의 차이

계획에 적힌 건수는 대개 짐작이다. 실제로 실행해 보면 그 짐작이 맞을 때도 있고 크게 어긋날 때도 있다. 어긋나면 저장소는 잘못된 전제로 절차를 골랐다는 뜻이므로, 좋은 계획이 나오지 않는다.

짐작이 어긋나는 흔한 이유는 자료의 분포가 고르지 않기 때문이다. 어떤 값은 열 건이고 어떤 값은 십만 건인데 평균으로 짐작하면 둘 다 틀린다. 또 다른 이유는 저장소가 참고하는 통계가 오래되어 지금 자료와 다른 경우다.

그래서 계획을 볼 때는 짐작한 건수와 실제 건수를 함께 본다. 둘이 크게 다른 단계가 있으면 그 단계가 문제의 출발점이다. 통계를 갱신하거나 조건을 바꿔 짐작이 맞게 만드는 것이 색인을 더하는 것보다 효과가 클 때가 많다.

3.4 계획이 바뀌는 이유

같은 조회가 어제는 빨랐는데 오늘 느리다면 계획이 바뀌었을 수 있다. 계획은 자료의 상태에 따라 정해지므로, 자료가 늘거나 분포가 달라지면 저장소가 다른 절차를 고른다.

값에 따라 계획이 달라지기도 한다. 조건에 흔한 값이 들어오면 전체를 훑는 편이 싸고, 드문 값이 들어오면 색인을 쓰는 편이 싸다. 저장소가 이 판단을 하면 같은 조회문이 값에 따라 다르게 처리된다. 1.2에서 본 긴 꼬리가 여기서 생기기도 한다.

계획이 바뀌는 것 자체는 정상이다. 문제는 바뀐 것을 모르는 것이다. 느린 조회를 기록할 때 그때의 계획도 함께 남겨 두면 이 변화를 나중에 확인할 수 있다. 남기지 않으면 재현되지 않는 느낌으로 남는다.

같은 조회의 두 계획

같은 조회문이 늘 같은 절차로 처리되지 않는다

흔한 값으로 물었을 때

드문 값으로 물었을 때

위로 갈수록
나중 단계

정렬

정렬 없음

다룬 건수 없음

거르기

자료 읽기

전체 훑기

색인 찾기

막대는 그 단계에서 다룬 건수다

중간에서 다룬 건수가 시간을 정한다

3.6 계획을 바꾸는 방법

계획이 마음에 들지 않을 때 쓸 수 있는 수단은 몇 가지다. 첫째는 색인을 조회에 맞게 만드는 것이다. 대부분의 계획 문제는 쓸 만한 색인이 없어서 생긴다.

둘째는 조회를 바꾸는 것이다. 조건을 가공하지 않고 그대로 걸거나, 필요 없는 정렬을 빼거나, 여러 자료를 묶는 순서를 바꾸면 계획이 달라진다. 셋째는 저장소가 참고하는 통계를 갱신해 짐작이 맞게 만드는 것이다.

저장소에 절차를 직접 지시하는 방법도 있지만 마지막에 쓴다. 지시한 절차는 자료가 달라져도 그대로 남아 나중에 더 나쁜 선택이 될 수 있다. 저장소가 스스로 좋은 계획을 고르게 만드는 편이 오래 간다. 어떤 수단을 썼든 바꾼 뒤에는 실제 시간을 다시 재어 확인한다.

3.7 계획만 보고 판단할 때

계획은 절차를 알려 주지만 실제 시간을 알려 주지는 않는다. 계획상 좋아 보이는 절차가 실제로는 느릴 수 있고, 그 반대도 있다. 짐작이 어긋나 있으면 계획의 비용 수치 자체가 믿을 것이 못 된다.

특히 놓치기 쉬운 것은 자료를 읽어 오는 실제 비용이다. 같은 건수라도 자료가 흩어져 있으면 모아 있을 때보다 오래 걸린다. 이 차이는 계획에 잘 드러나지 않는다.

그래서 계획은 실제 측정과 함께 본다. 계획으로 어디를 볼지 정하고, 시간으로 그 판단이 맞았는지 확인한다. 계획만 보고 고치면 좋아 보이는 절차로 바꿨는데 시간은 그대로인 일이 생긴다. 계획은 어디를 볼지 정해 주고 시간은 그 판단이 맞았는지 알려 준다.

4.1 한 번이 여러 번이 되는 자리

성능 문제 가운데 가장 흔하고 가장 크게 고쳐지는 것이 반복 조회다. 목록을 한 번 가져온 뒤 각 건마다 다시 조회하면, 목록이 백 건일 때 조회가 백한 번 일어난다.

이 구조가 문제인 이유는 자료가 늘수록 횟수가 함께 늘기 때문이다. 시험 환경에서 열 건일 때는 열한 번이라 눈에 띄지 않지만, 실제로 천 건이 되면 천한 번이 된다. 조회 하나하나는 여전히 빠르므로 느린 조회를 찾는 방식으로는 잡히지 않는다.

그래서 반복 조회는 시간이 아니라 횟수로 찾는다. 요청 한 번에 조회가 몇 번 일어났는지 세면 곧바로 드러난다. 이 셈이 없으면 각각이 빠른 조회들 사이에 숨어 오래 남는다. 횟수는 시간보다 세기 쉬우므로 이 셈을 먼저 갖추는 편이 빠르다.

4.2 목록과 상세의 관계

반복 조회는 대개 목록 화면에서 생긴다. 목록의 각 줄에 다른 자료가 필요할 때, 줄마다 그 자료를 가져오는 방식으로 짜기 쉽기 때문이다. 코드로는 자연스럽게 읽기도 쉽다.

이 구조를 바꾸는 방향은 둘이다. 하나는 목록을 가져올 때 필요한 자료를 함께 가져오는 것이고, 다른 하나는 목록을 먼저 가져온 뒤 필요한 자료를 한 번에 모아 가져오는 것이다. 앞쪽은 조회가 한 번이지만 결과가 커질 수 있고, 뒤쪽은 조회가 두 번이지만 각각이 단순하다.

어느 쪽이 나은지는 자료의 모양에 따라 다르다. 한 줄에 딸린 자료가 적으면 함께 가져오는 편이 낫고, 여러 건이 딸려 결과가 크게 부풀면 나눠 가져오는 편이 낫다. 중요한 것은 줄마다 조회하지 않는다는 원칙이다.

4.3 한꺼번에 가져오기

줄마다 조회하던 것을 하나로 합치는 방법은 단순하다. 필요한 식별자를 모두 모아 한 번에 조건으로 걸고, 받은 결과를 코드에서 나눠 붙인다. 조회 횟수가 건수와 무관해진다.

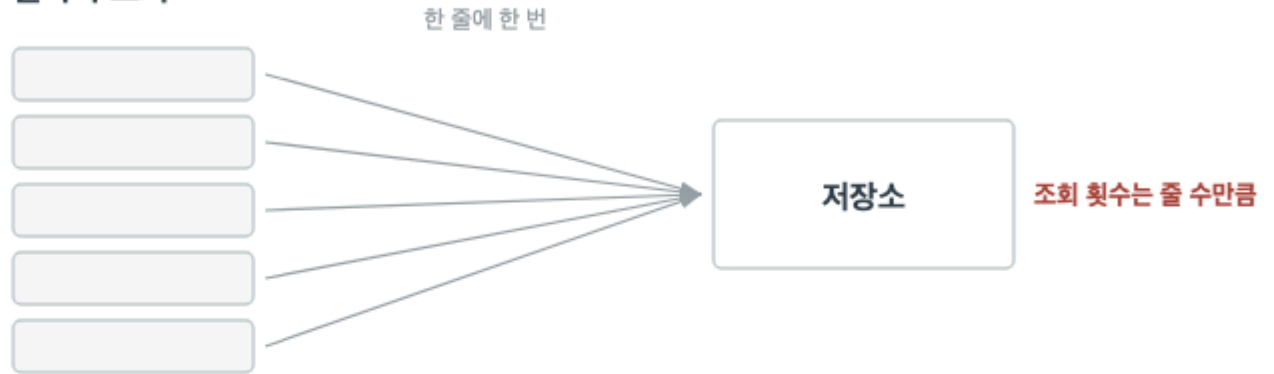
합칠 때 조심할 것은 한 번에 거는 식별자의 수다. 수천 개를 한 번에 걸면 조회 자체가 무거워지고 저장소가 계획을 세우기도 어려워진다. 그래서 적당한 크기로 나눠 몇 번에 걸쳐 가져오는 방식을 쓴다. 천 건을 한 번이 아니라 백 건씩 열 번으로 나누는 식이다.

합친 뒤에는 결과를 붙이는 코드가 생긴다. 이 코드는 조회 하나가 줄어드는 대가이며, 대개 그만한 값을 한다. 다만 붙이는 규칙이 복잡해지면 읽기 어려워지므로 이 부분을 한자리에 모아 두는 편이 낫다.

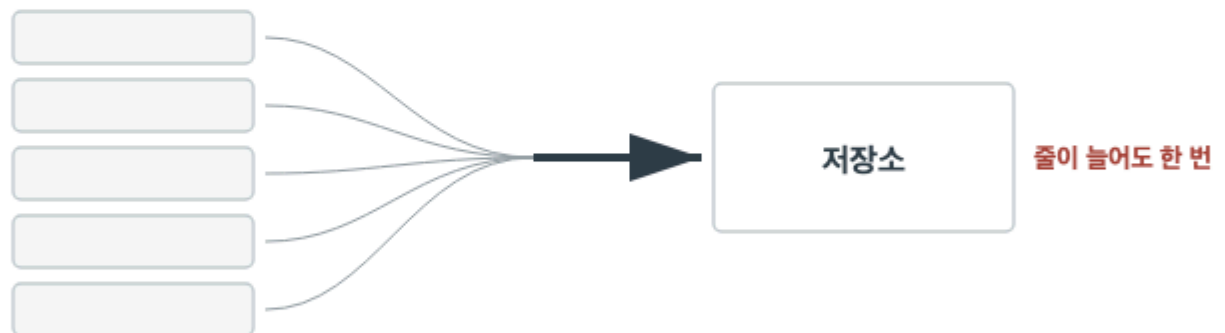
반복 조회가 늘어나는 모양

줄마다 묻는가 한 번에 묻는가

줄마다 조회



모아서 한 번



줄 수가 늘 때 달라지는 것은 시간이 아니라 횟수다

4.5 필요한 만큼만 가져오기

조회를 합치는 것만큼 자주 효과를 내는 것이 가져오는 양을 줄이는 일이다. 항목 전부를 가져오는 습관은 편하지만, 큰 항목이 섞여 있으면 옮기는 시간이 크게 늘어난다.

건수도 마찬가지다. 화면에 스무 줄만 보이는데 전부를 가져와 코드에서 자르는 방식은 자료가 늘면 곧바로 문제가 된다. 자를 것은 저장소에서 자르고 필요한 만큼만 받는다.

다만 줄이는 데도 균형이 있다. 항목을 지나치게 나눠 가져오면 조회가 여러 번이 되어 앞 절의 문제가 생긴다. 한 번에 필요한 것을 다 받되 필요 없는 것은 빼는 것이 기준이다. 무엇이 필요한지는 화면과 코드를 함께 보아야 알 수 있다. 가져오는 양을 줄이는 일은 저장소와 부르는 쪽 양쪽의 시간을 함께 줄인다.

4.6 캐시를 두기 전에

느린 조회를 만나면 결과를 어딘가에 보관해 두고 다시 쓰는 방법이 떠오른다. 효과는 크지만 새 문제를 함께 들어온다. 보관한 값이 언제까지 맞는지, 원본이 바뀌면 어떻게 할지를 정해야 한다.

그래서 보관을 고르기 전에 먼저 확인할 것이 있다. 이 조회가 왜 느린지 알고 있는가, 반복 조회나 빠진 색인처럼 구조에서 오는 문제는 아닌가다. 구조 문제를 보관으로 덮으면 원인이 남은 채로 복잡도만 는다.

보관이 맞는 경우도 있다. 자주 읽히고 거의 바뀌지 않으며 계산이 비싼 자료가 그렇다. 세 조건이 모두 맞을 때만 보관을 고르고, 하나라도 어긋나면 다른 방법을 먼저 본다. 보관을 고르면 그때부터 언제 버릴지가 새로운 설계 문제가 된다.

4.7 반복을 세는 방법

반복 조회를 찾으려면 요청 하나에 조회가 몇 번 일어났는지 세야 한다. 이 수는 시간과 달리 세기 쉽고, 한 번 세게 만들어 두면 계속 쓸 수 있다.

세는 자리는 조회를 실제로 보내는 지점이다. 그 지점에서 요청마다 수를 올리고 요청이 끝날 때 함께 남긴다. 수가 어느 선을 넘으면 눈에 띄게 표시해 두면 새로 생긴 반복 조회를 곧바로 알아챈다.

세는 것만으로 문제의 절반이 풀린다. 어떤 화면에서 조회가 수백 번 일어난다는 사실을 알면 고칠 자리는 대개 분명하다. 반대로 세지 않으면 그 화면은 그저 조금 느린 화면으로 남고, 자료가 늘 때마다 조금씩 더 느려진다. 세는 장치는 한 번 만들면 다른 화면에서도 그대로 쓰인다.

5.1 무엇을 관찰할 것인가

성능 문제를 예방하려면 자료를 어떻게 쓰고 있는지 알아야 한다. 어떤 조회가 자주 오는지, 어떤 조건으로 거르는지, 결과를 어떤 순서로 요구하는지가 접근 패턴이다.

패턴을 알면 색인을 짐작으로 만들지 않아도 된다. 자주 오는 조건에 맞춰 만들고, 오지 않는 조건에는 만들지 않는다. 색인은 조회 방식에 맞춰 만드는 것이므로 조회 방식을 모르면 만들 근거도 없다.

패턴은 시간이 지나면서 달라진다. 새 화면이 생기고 자료가 늘면 자주 오는 조회도 바뀐다. 그래서 관찰은 한 번 하고 마는 일이 아니라 이따금 다시 하는 일이다. 이 장은 무엇을 어떻게 이따금 볼지 다룬다. 관찰이 없으면 색인도 조회도 짐작으로 손대게 된다.

5.2 느린 조회 모으기

가장 먼저 갯출 관찰 장치는 오래 걸린 조회를 따로 남기는 것이다. 기준 시간을 정해 두고 그보다 오래 걸린 조회만 남기면 양이 감당할 만하면서도 볼 것은 다 남는다.

남길 때는 걸린 시간만이 아니라 조회의 모양, 그때의 조건 값 범위, 다룬 건수를 함께 남긴다. 시간만 남기면 다시 재현할 수 없고, 재현하지 못하면 계획을 볼 수도 없다.

기준 시간은 처음에 넉넉히 잡고 줄여 간다. 처음부터 낮게 잡으면 대부분의 조회가 걸려 아무것도 가려내지 못한다. 목록이 짧아질 때까지 기준을 올렸다가, 위쪽을 고치고 나서 다시 내리는 방식이 실용적이다. 기준을 낮출 때마다 새 후보가 드러나므로 개선은 단계로 이어진다. 모아 둔 목록은 다음 진단에서 어디부터 볼지 정하는 재료가 된다.

5.3 자주 오는 조회

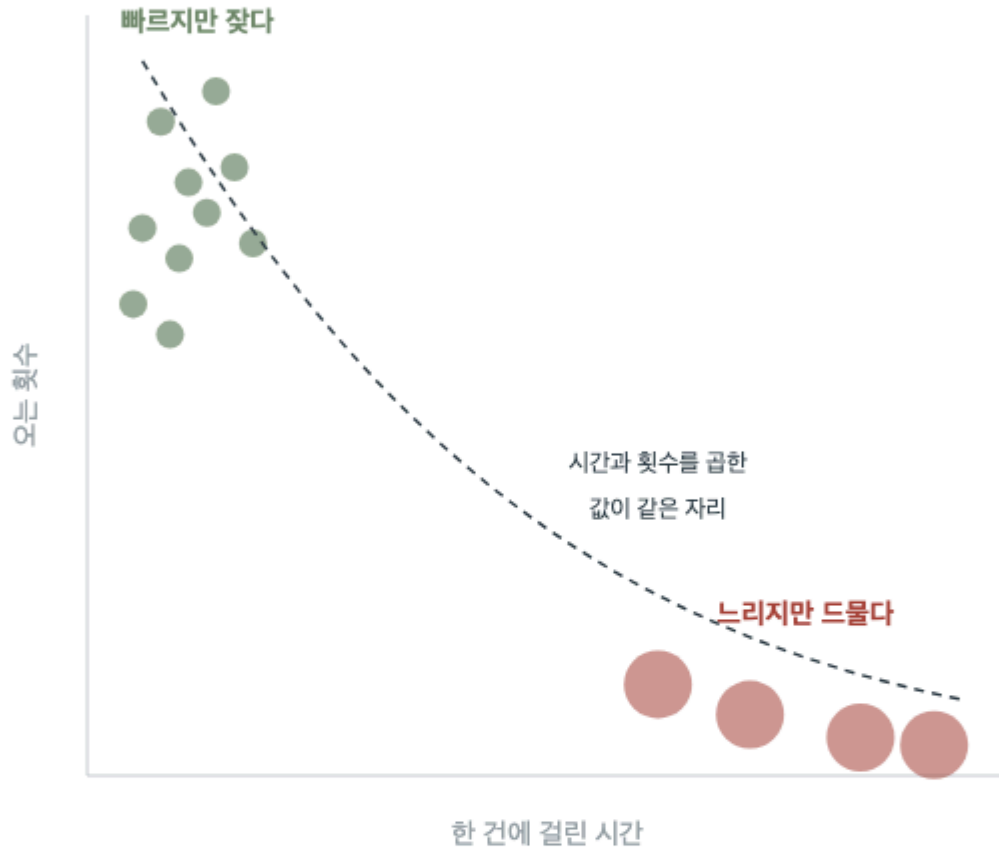
느린 조회만 보면 놓치는 것이 있다. 한 건에 아주 짧게 걸리지만 하루에 수백만 번 오는 조회다. 이런 조회는 목록의 위쪽에 오르지 않지만 저장소의 부담에서는 큰 몫을 차지한다.

그래서 조회를 모양별로 묶어 총 시간으로도 본다. 걸린 시간에 횟수를 곱한 값이 큰 순서로 늘어놓으면 느린 조회 목록과 다른 얼굴이 나온다. 실제로 줄일 것이 많은 쪽은 대개 이쪽이다.

모양별로 묶으려면 조건 값이 다른 조회를 같은 것으로 볼 수 있어야 한다. 값을 빼고 형태만 남겨 묶으면 같은 조회의 여러 실행이 하나로 모인다. 이 묶기가 안 되면 목록이 수천 줄이 되어 읽을 수 없다. 묶어 세는 일이 되면 어느 조회를 고쳐야 전체가 줄어드는지가 수로 보인다.

조회의 두 갈래 분포

오래 걸리는 것과 많이 오는 것은 다르다



곱한 값이 큰 쪽부터 손댄다

5.5 자료가 늘면 달라지는 것

지금 빠른 조회가 앞으로도 빠르다는 보장은 없다. 자료가 늘면 전체 훑기는 그만큼 느려지고, 계획이 바뀌기도 하며, 반복 조회의 횟수도 는다.

그래서 조회를 볼 때 자료가 열 배가 되면 어떻게 되는지 함께 묻는다. 답이 열 배 느려진다면 그 조회는 지금 빠르더라도 손볼 후보다. 답이 거의 그대로라면 지금 조금 느려도 급하지 않다.

이 물음은 시험 환경에서 답하기 어렵다. 시험 자료는 대개 적기 때문이다. 그래서 자료의 양을 늘려 보는 확인을 따로 두거나, 실제 자료의 증가 속도를 보고 언제쯤 문제가 될지 짐작한다. 늦게 알수록 고치는 비용이 크다. 지금의 속도보다 늘어남에 대한 성질이 더 오래가는 정보다. 늘어남에 비례하는 문제는 나중에 고칠수록 자료가 많아 고치기 어렵다.

5.6 실제와 시험 환경의 차이

성능 문제는 시험 환경에서 잘 드러나지 않는다. 자료가 적고, 동시에 오는 요청이 없고, 자료의 분포도 다르다. 실제에서만 느린 조희가 흔한 이유다.

차이를 줄이는 방법은 자료의 양과 분포를 비슷하게 만드는 것이다. 양만 늘리고 분포를 맞추지 않으면 여전히 다르다. 실제에서는 한 값에 십만 건이 몰려 있는데 시험에서는 고르게 퍼져 있으면 계획부터 달라진다.

차이를 다 없앨 수는 없으므로 실제에서 관찰하는 장치를 함께 갖춘다. 느린 조희를 모으고 조희 횟수를 세는 장치는 실제 환경에서 가장 값을 한다. 시험은 미리 거르는 자리이고 실제는 확인하는 자리다. 두 자리의 차이를 적어 두면 시험을 통과한 뒤에도 무엇을 더 볼지 알 수 있다.

5.7 관찰이 부담이 될 때

관찰에도 비용이 든다. 모든 조회의 시간을 재고 계획을 남기면 그 자체가 부하가 된다. 특히 조회가 잦은 자리에서는 재는 비용이 조회 비용과 비슷해지기도 한다.

그래서 관찰은 늘 켜 두는 것과 필요할 때 켜는 것으로 나눈다. 조회 횟수를 세는 일은 가벼우므로 늘 켜 두고, 계획을 남기는 일은 무거우므로 진단할 때만 켜다. 이 구분이 없으면 관찰을 아예 꺼 버리게 되고 그러면 아무것도 모르게 된다.

표본을 쓰는 방법도 있다. 백 건에 한 건만 자세히 남기면 부하는 백분의 일이고 경향은 그대로 보인다. 다만 드문 문제는 표본에 잡히지 않으므로, 느린 조회는 표본과 무관하게 모두 남기는 편이 낫다. 관찰을 끄는 선택은 부하를 줄이지만 다음 진단을 처음부터 다시 하게 만든다.

6.1 느린 화면의 증상

이 장은 느린 목록 화면 하나를 고친 기록이다. 처음 받은 말은 주문 목록 화면이 갈수록 느려진다는 것이었다. 정확한 시간은 아무도 재고 있지 않았다.

먼저 시간을 잰다. 요청 한 번이 끝나기까지 평균 사 초, 상위 몇 퍼센트는 십 초를 넘었다. 같은 화면인데 값에 따라 편차가 컸다. 목표는 사용자가 기다린다고 느끼기 전, 곧 일 초 아래로 잡았다.

다음으로 시간이 어디에 들어가는지 나눴다. 저장소에서 걸린 시간의 합이 전체의 대부분이었고, 부르는 쪽 처리 시간은 얼마 되지 않았다. 이 시점에서 볼 자리가 저장소 쪽으로 좁혀졌다.재기 전에는 모두가 다른 원인을 짐작하고 있었고 그 짐작은 서로 달랐다.그래서 짐작을 적어만 두고 수를 먼저 갖추기로 했다.

6.2 조회 횟수를 세어 본 결과

저장소에서 걸린 시간이 크다는 것은 알았지만 어떤 조회가 문제인지는 아직 몰랐다. 느린 조회 목록을 보았더니 이 화면과 관련된 조회는 하나도 올라 있지 않았다.

그래서 횟수를 셸다. 요청 한 번에 조회가 이백 번 넘게 일어나고 있었다. 목록이 오십 줄이었고 줄마다 네 번씩 조회가 나가는 구조였다. 각각은 몇 밀리초라 느린 조회 목록에 오르지 않았던 것이다.

이 수를 보고 나서야 문제의 모양이 정해졌다. 조회 하나를 빠르게 만드는 문제가 아니라 조회 횟수를 줄이는 문제였다. 시간만 보고 있었다면 색인을 손대며 시간을 썼을 것이다.세는 장치는 이 진단을 위해 만들었지만 이후 다른 화면에서도 쓰였다.이 수 하나가 남은 진단의 방향을 통째로 바꿨다.

6.3 반복을 줄인 과정

네 종류의 반복 조회 가운데 셋을 합쳤다. 줄마다 가져오던 자료를 식별자로 모아 한 번에 가져오고 코드에서 붙였다. 한 번에 거는 식별자가 오십 개라 나눌 필요는 없었다.

남은 하나는 합치기 어려웠다. 줄마다 조건이 달라 한 번의 조회로 모을 수 없는 형태였다. 이 조회는 화면에서 실제로 쓰이는지 확인했더니 화면 어디에도 표시되지 않는 값이었다. 조회 자체를 지웠다.

합치고 지우자 조회 횟수가 이백여 번에서 네 번으로 줄었다. 시간은 사 초에서 일 초 근처로 떨어졌다. 아직 목표에 달지는 않았지만 남은 문제의 모양이 달라졌다. 쓰이지 않는 조회 하나를 지운 것이 합치는 일보다 간단하고 효과도 컸다. 남은 시간은 조회 한 건의 문제였고 그것은 다음 절에서 다뤘다.

6.4 남은 조회의 실행 계획

남은 네 번 가운데 목록을 가져오는 조회가 시간의 대부분을 쓰고 있었다. 이 조회의 계획을 보았다.

계획에는 세 가지가 드러났다. 전체를 훑고 있었고, 중간에서 다룬 건수가 최종 결과의 수백 배였으며, 마지막에 정렬이 있었다. 조건은 상태와 기간 두 가지였는데 어느 쪽에도 쓸 만한 색인이 없었다.

짐작한 건수와 실제 건수도 크게 달랐다. 상태 값의 분포가 고르지 않아 한 값에 자료의 대부분이 몰려 있었는데, 저장소는 고르게 퍼져 있다고 짐작하고 있었다. 이 어긋남이 계획 선택을 흔들고 있었다. 계획을 보기 전까지는 색인이 없다는 사실조차 확인되지 않은 짐작이었다.여기까지 오는 데 코드는 거의 읽지 않았고 계획과 수만 보았다.

6.5 색인을 정한 과정

색인은 상태와 기간 두 항목을 묶어 만들되 순서를 정해야 했다. 상태는 등치로 걸리고 기간은 범위로 걸리므로 상태를 앞에, 기간을 뒤에 두었다.

정렬도 함께 보았다. 화면이 요구하는 순서가 기간 내림차순이라 색인의 순서와 맞출 수 있었다. 순서를 맞추자 계획에서 정렬 단계가 사라졌다. 색인 하나로 찾기와 정렬을 함께 해결한 것이다.

만들기 전에 이 색인이 쓰기에 얼마나 부담이 되는지 확인했다. 주문은 자주 들어오지만 이 두 항목은 거의 바뀌지 않아 갱신 비용이 작았다. 색인을 더한 이유와 대상 조회를 함께 적어 두었다. 만들 색인의 순서를 정하는 데 든 시간이 색인을 만드는 시간보다 길었다. 적어 둔 근거는 나중에 이 색인을 지울지 판단할 때 쓰인다.

6.6 고친 뒤 다시 쟀 결과

색인을 만든 뒤 다시 쟀다. 목록 조회는 계획이 바뀌어 색인을 쓰고 있었고, 중간에서 다루는 건수가 최종 결과와 비슷해졌다. 요청 전체는 평균 이백 밀리초 아래로 내려갔다.

꼬리도 함께 보았다. 상위 몇 퍼센트가 여전히 일 초를 넘고 있었다. 그 조회들을 모아 보니 기간을 아주 길게 잡은 요청들이었다. 이 경우는 가져오는 건수 자체가 커서 색인으로 줄일 수 없는 종류였다.

그래서 화면에서 기간의 상한을 두기로 했다. 성능 문제 가운데 일부는 저장소가 아니라 요구를 조정해 푸는 것이 맞다. 목표였던 일 초 아래를 상위 구간에서도 넘기지 않게 되었다. 고친 뒤의 수를 고치기 전과 같은 방식으로 재야 견줄 수 있다.

6.7 남긴 장치와 다음 후보

고친 뒤에 남긴 것은 두 가지다. 요청마다 조회 횟수를 세어 일정 수를 넘으면 표시하는 장치와, 이 화면의 응답 시간을 계속 보는 지표다. 둘 다 다시 나빠질 때 알아채기 위한 것이다.

느린 조회를 모으는 기준 시간도 낮췄다. 위쪽을 고치고 나니 목록이 짧아져 더 낮은 기준으로도 볼 만해졌다. 기준을 낮추자 다른 화면의 조회 둘이 새로 올라왔다.

그 둘을 다음 후보로 적어 두었다. 하나는 이번과 같은 반복 조회로 보이고 다른 하나는 색인이 없는 조회로 보인다. 짐작은 짐작으로 적어 두고, 실제로 손댈 때 다시 재기로 했다. 이번 진단에서 만든 장치가 다음 진단의 출발점이 되었다. 고친 자리보다 남긴 장치가 다음 사람에게 더 오래 쓰인다.

7.1 어디부터 손댈 것인가

고칠 후보가 여럿이면 순서를 정해야 한다. 기준은 걸린 시간에 횟수를 곱한 값이다. 이 값이 큰 쪽부터 손대면 같은 노력으로 더 크게 줄어든다.

다만 이 값이 큰 것이 늘 고치기 쉬운 것은 아니다. 그래서 실무에서는 곱한 값과 고치는 데 드는 품을 함께 놓고 고른다. 크고 쉬운 것, 크고 어려운 것, 작고 쉬운 것 순서가 대체로 맞다.

여기에 하나를 더 본다. 자료가 늘면 더 나빠지는 종류인가다. 지금은 작아도 늘어남에 비례하는 문제라면 일찍 손대는 편이 싸다. 반복 조회와 전체 훑기가 이 종류에 든다. 순서를 적어 두면 중간에 다른 일이 끼어들어도 이어서 할 수 있다. 무엇을 먼저 할지 정하는 일이 어떻게 고칠지 정하는 일보다 앞선다.

7.2 고치기 전에 재기

성능 작업에서 가장 자주 깨지는 규칙이 재고 나서 고치는 것이다. 짐작으로 색인을 더하고, 짐작으로 조회를 합치고, 나아졌으리라 여기며 끝낸다. 이때 실제로 나아졌는지 아는 사람이 없다.

재지 않고 고치면 세 가지를 잃는다. 무엇이 문제였는지, 고침이 얼마나 효과가 있었는지, 그리고 다음에 같은 자리가 나빠졌을 때 견줄 기준이다. 고치기 전에 재는 것으로는 부족하고, 고친 뒤에도 같은 방식으로 재야 견줄 수 있다.

재는 일은 대개 고치는 일보다 싸다. 조회 횟수를 세는 장치와 느린 조회를 모으는 장치는 한 번 만들면 계속 쓴다. 이 둘만 있어도 진단의 절반이 자동으로 된다. 재는 장치가 없는 상태에서 시작하는 성능 작업은 대개 짐작으로 끝난다.

7.3 색인을 더할 때와 지울 때

색인을 더하는 판단은 단순하다. 자주 오는 조회가 있고 그 조회가 지금 색인을 쓰지 못하며, 쓰기 부담이 감당할 만하면 더한다. 셋 가운데 하나라도 아니면 다른 방법을 먼저 본다.

지우는 판단은 더 어렵다. 쓰이지 않는 색인은 비용만 남기지만, 쓰이지 않는다는 것을 확인하기가 쉽지 않다. 그래서 색인마다 어떤 조회를 위한 것인지 적어 두고, 그 조회가 사라졌는지로 판단하는 방식이 실용적이다.

색인 목록은 시간이 지나면 늘기만 한다. 더할 때마다 근거를 적고 이따금 목록을 훑어 근거가 사라진 것을 지우는 절차를 두지 않으면, 몇 년 뒤에는 쓰기가 느린 이유를 아무도 설명하지 못한다. 색인 목록은 줄이는 절차를 두지 않으면 늘어나기만 하는 목록이다.

7.4 성능 점검 항목

느린 화면을 만나면 다음을 차례로 본다. 첫째, 요청 한 번에 조회가 몇 번 일어나는가. 수십 번을 넘으면 반복 조회를 먼저 본다.

둘째, 저장소에서 걸린 시간이 전체에서 얼마인가. 작으면 저장소 밖을 본다. 셋째, 그 조회의 계획이 전체를 훑고 있는가. 넷째, 중간에서 다룬 건수가 최종 결과보다 훨씬 크다. 크면 거르기를 앞으로 당길 수 있는지 본다.

다섯째, 가져오는 항목과 건수가 실제로 필요한 만큼인가. 걸리는 항목이 나오면 그 항목을 설명한 절로 돌아간다. 순서대로 보는 이유는 앞의 것이 대개 더 크게 줄고 더 쉽게 고쳐지기 때문이다. 다섯을 순서대로 보면 대개 두 번째나 세 번째에서 원인이 드러난다.

7.5 구조를 바꾸는 선택

조회와 색인을 손봐도 목표에 닿지 않을 때가 있다. 이때는 자료를 두는 방식이나 화면의 요구를 바꾸는 쪽을 본다. 미리 계산해 두는 방식, 결과를 보관해 다시 쓰는 방식, 요구 범위를 좁히는 방식이 있다.

셋 다 새 문제를 들여온다. 미리 계산한 값은 원본과 어긋날 수 있고, 보관한 값은 언제 버릴지 정해야 하며, 요구를 좁히는 것은 사용자가 할 수 있는 일을 줄인다. 그래서 앞의 방법을 다 써 본 뒤에 고른다.

고를 때는 어긋남을 어디까지 견딜 수 있는지 먼저 정한다. 잠깐 어긋나도 되는 화면이라면 미리 계산하거나 보관하는 방법이 맞고, 정확한 값이 늘 필요하다면 요구를 좁히는 쪽이 맞다. 어느 쪽을 골라도 왜 그렇게 했는지를 적어 두어야 나중에 되돌릴 수 있다.

7.6 성능과 다른 목표의 균형

성능을 위해 한 일은 대개 다른 것을 조금씩 나쁘게 만든다. 색인은 쓰기를 느리게 하고, 합친 조회는 코드를 복잡하게 하며, 보관한 값은 정확성을 혼든다. 공짜로 빨라지는 경우는 드물다.

그래서 목표를 넘겼는지로 멈출 자리를 정한다. 목표를 넘긴 뒤의 최적화는 이득이 작고 대가는 그대로 남는다. 성능은 높을수록 좋은 값이 아니라 정해 둔 선을 넘겼는지로 판단하는 값이다.

치른 대가는 적어 둔다. 왜 이 색인을 만들었는지, 왜 이 조회를 합쳤는지, 왜 이 값을 보관하는지를 남겨야 나중에 그 근거가 사라졌을 때 되돌릴 수 있다. 되돌릴 수 없는 최적화가 쌓이면 구조가 굳는다. 빠르게 만드는 일과 고치기 쉬운 상태를 지키는 일은 늘 함께 저울에 오른다.

7.7 핵심 원칙 정리

이 책이 반복한 물음은 둘이다. 이 시간이 어디에 쓰이는가, 그리고 자료가 늘면 어떻게 되는가. 앞의 물음이 지금 볼 자리를 정하고 뒤의 물음이 언제 손댈지를 정한다.

나머지는 두 물음의 도구다. 실행 계획은 시간이 어디에 쓰이는지 절차로 보여 주고, 색인은 자료가 늘어도 찾는 시간이 늘지 않게 하며, 조희 힛수 세기는 늘어남이 힛수로 나타나는 자리를 드러낸다.

끝으로 덧붙일 것은 성능 작업이 짐작으로는 되지 않는다는 점이다. 재고, 계획을 읽고, 다시 재는 절차를 밟으면 처음 보는 시스템에서도 같은 순서로 진단할 수 있다. 절차가 경험을 대신하지는 않지만 경험이 없어도 시작할 수는 있게 한다. 재고 읽고 다시 재는 순서만 지켜도 대부분의 성능 문제는 자리를 드러낸다.